Cost-Aware Model Router

A per-query difficulty classifier that decides between a local small model and a frontier API call. Target: **2–5× cost reduction at ≥95% of frontier quality** — with a threshold you can actually defend.

Owner Ani **Scope** 4 parts, 8 phases, ~8 days **Source of truth** DESIGN.md

REV A

Confirmed constraints, and the one move that makes them irrelevant

SUPERSEDES §1 DATASETS · §4 PHASE 2 & PHASE 8 · APPENDIX A — 26 AUG 2026

Local compute: Apple Silicon, ≥16 GB unified memory, no CUDA. · **Frontier API budget:** none. · **Goal:** academic report / thesis.

Phase 2 was the only irreversible spend in the plan above. With no budget it is replaced by *acquisition* — and the project loses nothing. It gains statistical power.

TRACK A · PRIMARY · $0

RouterBench as the substrate

RouterBench released **405,467 precomputed inference outcomes** — 11 models × 8 datasets × 64 tasks, with per-item quality scores and token costs already computed.

- Models include GPT-4, GPT-3.5-turbo, Claude-v1/v2/instant, Llama-70B-chat, Mixtral-8x7B, Yi-34B, Mistral-7B-chat.
- Datasets: **GSM8K, MMLU, MT-Bench, MBPP**, ARC-C, HellaSwag, Winogrande, RAG.
- Suggested pair: `S` = Mistral-7B-chat, `L` = GPT-4. Claude-v2 as an alternate `L` for a robustness column.

TRACK B · VALIDATION · $0 MARGINAL

A real local model, on your machine

Shows the router works with a model you actually control, not only with logged responses. `S` = **Qwen3-8B at 4-bit via MLX** — prefer MLX over Ollama on Apple Silicon; expect roughly 25–40 tok/s.

- **MMLU is nearly free locally** — four forward passes per item for logprob scoring, not generation. 2,000 items in minutes. Do this first.
- The `L` arm reuses RouterBench's GPT-4 per-item scores on overlapping items; where ids don't align, say so.
- Runs as a background job **in parallel with Phases 4–6** — not on the critical path.

Three things Track A buys you

1. **Re-pricing is the contribution.** RouterBench's costs are 2024-era API prices. Recompute `c_L` from its token counts at current 2026 rates (Sonnet 4.6 at $3/$15) and — the key move — set **`c_S = 0`**, because in your framing the weak arm runs locally. *"What does the cost–quality frontier look like when the weak arm is free?"* is not a question the original paper asked.
2. **`n_cal` stops being the binding constraint.** §3 showed the confidence bound's width is set by `n_cal`, and that 1,000–2,000 per dataset was the requirement. RouterBench hands you that for free. **The statistical rigor becomes your contribution rather than your caveat.**
3. **MT-Bench arrives pre-judged**, which removes leakage vector 4 — judge self-preference — entirely for Track A.

THE NEW BOTTLENECK IS TIME, NOT MONEY

GSM8K in Track B is the expensive item: 1,500 items × k=3 × ~250 output tokens ≈ 1.1M tokens ≈ **8–12 hours** at 30 tok/s. Plan it as one resumable overnight run with `k=3` and `max_tokens=320`. MMLU, by contrast, is minutes.

THE $5–10 UPGRADE — FLAG IT IN THE REPORT

If any budget appears: 1,000 GSM8K + 1,000 MMLU items through **Sonnet 4.6 on the Batch API** costs roughly **$2–3 at k=1** and **$7–10 at k=3** (batch rate $1.50/$7.50 per MTok; GSM8K ≈ 120 in / 250 out per item, MMLU ≈ 200 in / 5 out). That converts Track B into a genuine, current, Anthropic-model result — the highest-value marginal dollar in the whole project.

Revised phases

PHASE	CHANGE
2 → 2′	**Generation becomes acquisition.** Download and normalize RouterBench, verify the per-item score and cost fields, map to the unified schema. ~0.5 day, $0.
3	Kill-check still applies — but now you **choose the pair by headroom**, maximizing `Q_L − Q_S`. That was impossible when you were paying to generate.
4	**F3 moves from stretch to feasible.** With 405k rows you are no longer data-limited, and DeBERTa-v3-small trains overnight on MPS. Worth one run as the top ablation row.
8	**Dropped.** Thesis target, no serving requirement.

Thesis framing — three defensible contributions

1. A re-analysis of RouterBench under a **free weak arm** and **current 2026 frontier prices**.
2. **Distribution-free threshold selection** (Learn-then-Test) applied to router deployment — the field reports point estimates; you report a bound.
3. A **random/oracle-bracketed ablation ladder** (F0→F1→F2→F3, decision rule v1 vs cost-normalized v2) across both grouped and leave-one-subject-out splits.

00 Problem, framing, and the three decisions that matter

Three calls determine whether this project produces a real result or a plausible-looking one. Everything else is execution detail.

DECISION 01

The asset is the paired response corpus, not the classifier

For every query you need *both* models' answers and a comparable score. That is the one expensive, irreversible step. Generate it once, content-hash cache it, freeze it — then features, classifier and threshold sweep are all cheap and re-runnable.

DECISION 02

Predict the quality gap, not the winner

Two calibrated heads predict `ŝ_S` and `ŝ_L` separately; the routing score is their difference. This handles "both models fail" correctly, survives swapping either endpoint, and extends to a three-tier cascade without redesign.

DECISION 03

τ selection is statistics, not plotting

A threshold chosen on the same data used to report savings is optimistically biased. Three-way split plus a *paired* bootstrap lower confidence bound. This is the largest source of fake numbers in routing work and the cheapest place to be more rigorous than the field.

Formalization

Let `q` be a query, `S` the small local model, `L` the frontier API model. Quality `s_M(q) ∈ [0,1]` under a fixed grader; cost `c_M(q)` in dollars. A router `r_τ` is parameterized by one scalar.

```
Q(τ) = E_q[ s_{r_τ(q)}(q) ]          realized quality
C(τ) = E_q[ c_{r_τ(q)}(q) ]          realized cost

minimize C(τ)   subject to   Q(τ) ≥ Q_target
```

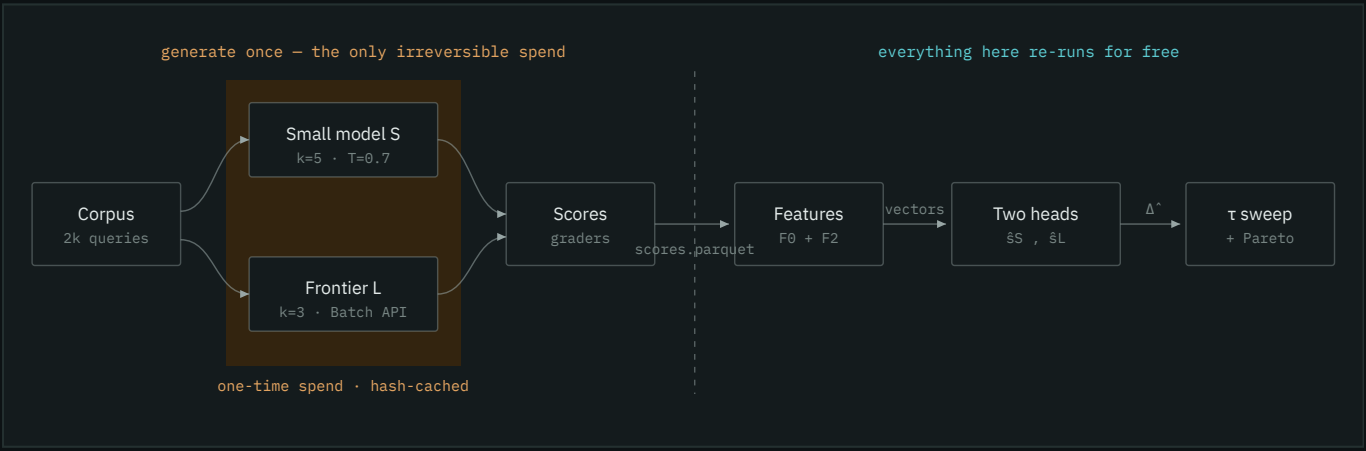
"95% of frontier quality" is ambiguous — pick one now

READING	TARGET	CONSEQUENCE
Absolute	`Q ≥ 0.95 · Q_L`	Weak. If `Q_S` already exceeds 0.95· `Q_L` on a dataset, routing 100% local satisfies it and the number is meaningless.
PGR `recommended`	`Q ≥ Q_S + 0.95·(Q_L − Q_S)`	Strict — recover 95% of the *headroom* the frontier buys you. Standard in the literature; makes results comparable to RouteLLM.

Headline on PGR, report absolute as a secondary column. The two can differ by tens of percentage points of spend.

ROUTER COST BUDGET

The router must be negligible next to `S` : **< 15 ms p99 on CPU, < 100 MB RAM**. That constraint is why the feature ladder stops at a frozen sentence encoder with an MLP head, and why a 7B "router LLM" is out of scope by construction.



Where the money goes. Only the shaded block costs anything. Once `scores.parquet` crosses the dashed boundary it is frozen, and every feature set, classifier and threshold you try afterwards is free. Build the repo so that boundary is enforced by a content-hash cache, not by discipline.

01 Dataset & feature engineering strategy

Dataset selection

> SUPERSEDED BY REV A
>
> With no API budget, these datasets are sourced from **RouterBench** rather than generated — same benchmarks, precomputed scores. The reasoning below still explains *why* these three regimes matter; see Rev A for what you actually download.

Span two regimes: **verifiable** (objective, cheap, noiseless labels) and **realistic** (actual user traffic distribution).

ID	DATASET	N USED	GRADER	WHY THIS ONE
D1	**GSM8K**	1,500	exact match, numeric	Difficulty is genuinely bimodal for 3–8B models. Objective labels, no judge needed.
D2	**MMLU**	2,000	logprob argmax A/B/C/D	57 subjects. The "which topics does S fail at" signal is the most learnable routing feature that exists.
D3	**LMSYS Chatbot Arena**	1,500	LLM judge, position-swapped	The *only* source of a realistic query distribution. Production savings are set by the query mix, not by benchmark accuracy.
D4	MBPP / HumanEval `optional`	400	sandboxed execution	Execution grading is the cheapest reliable signal available. Include if code is in the deployment target.

> REPORTING RULE
>
> Never average across D1–D3. Different graders, different natural cost profiles. Report a Pareto curve *per dataset* plus a blended-traffic curve with the mixture weights stated explicitly.

Label construction THIS DOMINATES CLASSIFIER QUALITY

A single greedy sample gives a 0/1 Bernoulli label with enormous variance. Use soft labels instead — empirically worth more than any feature upgrade:

- **GSM8K / MBPP** — sample `k=5` from `S` at `T=0.7`, label `s_S = #correct/5 ∈ {0,.2,.4,.6,.8,1}`. `k=3` from `L`. Cuts label variance roughly 5×.
- **MMLU** — don't sample at all. Score the four options by length-normalized log-likelihood and use `P(correct)` as a soft score. Free, low-variance. Report hard accuracy separately.
- **Arena** — judge `S` vs `L` in *both* orderings; `s_S ∈ {0, 0.5, 1}`. Treat swap-disagreements as ties and report the disagreement rate as a judge-reliability number.

Freeze everything. A `manifest.json` beside the corpus records model version strings, decode params, prompt-template hash, seeds, grader version, and total spend. Any change invalidates cross-run

comparison.

The feature ladder

Implement in order. Each tier is the baseline for the next — the *deltas* are the interesting result, not the final number.

TIER	REPRESENTATION	MODEL	WHAT IT BUYS YOU
F0	~25 hand-built heuristics: token count, digit density, code fences, LaTeX, bracket depth, Flesch-Kincaid, "prove / derive / step by step" cue counts	Logistic regression	Proves signal exists, and gives a free interpretable baseline. Token count alone is a surprisingly strong router.
F1	TF-IDF: word 1–2 grams **concatenated with char 3–5 grams**	LogReg / LinearSVC	Char n-grams matter here — they capture math notation and code syntax that word tokens miss. Expect this to get most of the way on D1/D2.
F2	`bge-base-en-v1.5` (768d) frozen, **concatenated with F0**	MLP head	`headline config` The combination beats either alone: length and complexity signals are poorly represented in semantic embedding space.
F3	ModernBERT-base / DeBERTa-v3-small, fine-tuned end-to-end	cross-encoder	Highest ceiling. Needs a GPU and ≳10k labels. Only pursue if F2 is data-limited rather than signal-limited.
F4	Mean token logprob, entropy, max-margin over `S` 's first 32 generated tokens	appended to F2	Usually the single strongest feature — but it makes routing non-free. Frame it as a **cascade** variant and price the wasted prefix into `C(τ)` .

02 Router classifier architecture & strategy

Objective function — three candidates, one winner

OPTION	TARGET	VERDICT
(a) Binary success of S	`y = 1[s_S(q) ≥ θ]`	Rejected — ignores whether `L` also fails, so it escalates queries nobody can solve. Pure wasted spend.
(b) Relative gap regression	`y = s_L(q) − s_S(q)`	**Chosen.** Models exactly what escalation buys. Handles both-fail (gap≈0 → stay local) and both-succeed (gap=0 → stay local).
(c) End-to-end cost-sensitive	`min E[loss] + λ·E[cost]`	Rejected — bakes `λ` into the weights, so you lose the ability to sweep `τ` post hoc and re-derive the whole Pareto curve for free. Also unstable at this data scale.

Two-head architecture

```
x   = [ encoder(q) ‖ heuristics(q) ]     # frozen, 768 + 25 dims
h   = MLP(x)                             # 256-d shared trunk
ŝ_S = σ(w_S · h + b_S)                   # predicted quality of small model
ŝ_L = σ(w_L · h + b_L)                   # predicted quality of frontier model
Δ̂(q) = ŝ_L(q) − ŝ_S(q)                   # routing score

loss = BCE(ŝ_S, s_S) + BCE(ŝ_L, s_L) + β · MSE(ŝ_L − ŝ_S, s_L − s_S),  β ≈ 0.5
```

Soft-label BCE is valid for targets in `[0,1]` and better behaved than MSE on bounded scores; the auxiliary gap term keeps the *difference* well-fit even when both marginals are easy. Two heads rather than one gap head because: swapping the frontier model retrains only `w_L`, adding a middle tier is one more head, and each head is individually interpretable and individually calibratable.

Decision rule

v1 (baseline) — route to `L` iff `Δ̂(q) > τ`.

> **v2 (recommended) — cost-normalized gain**
>
> `G(q) = Δ̂(q) / ( ĉ_L(q) − ĉ_S(q) )` route to L iff `G(q) > τ`

Escalating a 4,000-token query costs far more than a 200-token one, so the naive rule over-spends on long queries. `G` is "expected quality points per dollar," and thresholding it is the Lagrangian-optimal rule for the constrained problem. Expect a visibly better frontier wherever query lengths are heterogeneous — which they are in Arena data. Ship v1 first; the v1→v2 delta is a genuine contribution. `ĉ_L(q)` needs a predicted output length: a third tiny head regressing `log T_out` on the frozen corpus does the job.

Calibration

Raw head outputs are not calibrated, and `τ` must transfer across datasets. Fit **isotonic regression** per head on the calibration split, mapping `ŝ_M → E[s_M | ŝ_M]`. Isotonic over Platt because the miscalibration here is non-sigmoidal at the extremes. Deliverable: reliability diagrams and 15-bin ECE, before and after.

Training setup

Encoder	frozen; embeddings precomputed once into `features.parquet`
Head	793 → 256 (GELU, dropout 0.1) → 2
Optimizer	AdamW, lr 1e-3 head-only / 2e-5 if fine-tuning, weight decay 0.01, cosine schedule, 10% warmup
Batch / epochs	64 / 30 with early stopping, patience 5
Model selection	validation **Spearman ρ(Δ̂, Δ)** — *not* accuracy. Ranking quality is what the threshold sweep consumes.
Imbalance	if `S` succeeds >80% of the time, weight samples by `|Δ|`. Always report per-bucket error.
Seeds	5; mean ± std on every headline number

Split logic — four leakage vectors

This section is the difference between a believable result and a fake one.

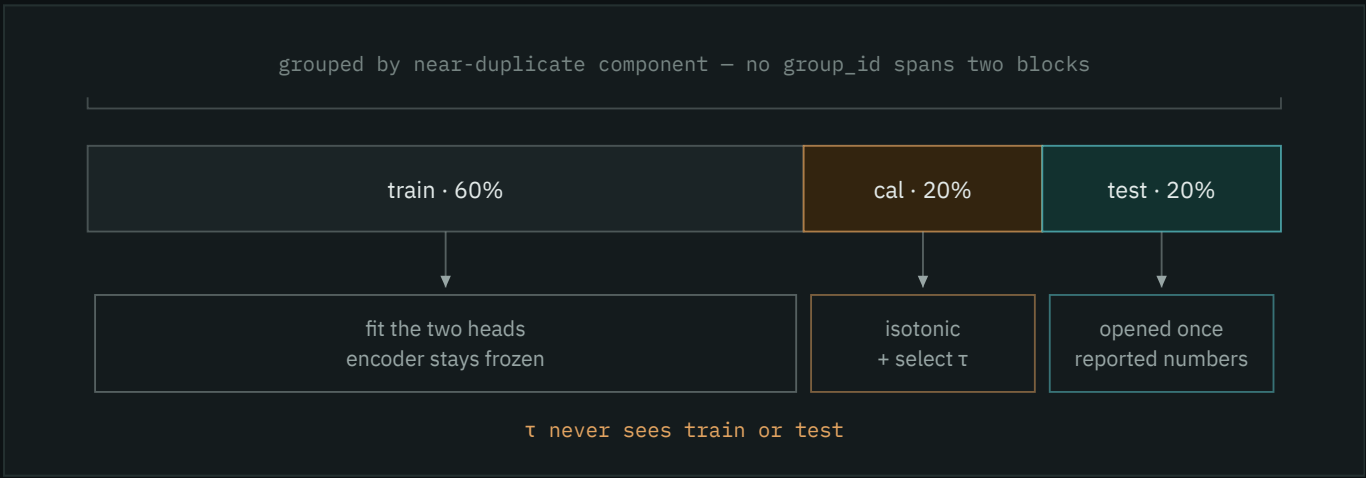
1 · NEAR-DUPLICATE LEAKAGE

GSM8K contains templated and paraphrased problems; Arena contains repeated prompts. A near-duplicate straddling train and test hands the router an answer key. **Fix:** before splitting, run MinHash LSH on 5-gram shingles (Jaccard ≥ 0.8) *and* embedding-cosine dedup (≥ 0.95). Build a connected-components graph over near-duplicates and assign each *component* one `group_id`. Split by group, never by row. Emit the dedup report as an artifact.

2 · GROUP / DOMAIN LEAKAGE

With a random split, the router memorizes `subject → difficulty` across MMLU's 57 subjects. That is legitimate signal for in-domain deployment but badly inflates out-of-domain estimates. **Fix:** report both a random grouped split *and* a leave-one-subject-out split. The gap between them is your generalization headroom, and it is one of the more interesting results in the writeup.

3 · THRESHOLD LEAKAGE — THE BIG ONE



The only split that matters. Choosing `τ` on the same data you report it against guarantees optimistic savings. `n_cal ≥ 1,000` is a hard floor — the width of the confidence bound in §3 is set by `n_cal`, not by `n_train`.

4 · JUDGE / LABEL LEAKAGE

If the LLM judge is the same family as `L`, self-preference bias inflates `s_L` and therefore inflates the measured headroom. **Fix:** a judge from a different family than both routed models where possible, always position-swapped, and report judge–human agreement on the Arena subset where human preference labels exist. If no cross-family judge is available, state the bias and treat Arena PGR as an upper bound.

> SMALL-DATA PROTOCOL
>
> If the corpus is under 5k after dedup, replace the single split with **nested grouped CV**: 5 grouped outer folds for reporting, an inner fold within each training portion for `τ`. Report mean ± std across outer folds.

03 Metric & Pareto curve evaluation pipeline

Quality metric

DATASET	GRADER	SCORE
GSM8K	numeric normalization (strip `$`, commas, trailing `.0`), exact match on the final answer	`pass@1 over k=5`
MMLU	length-normalized logprob argmax over A/B/C/D	`P(correct), soft`
Arena / MT-Bench	pairwise LLM judge, both orderings	`win 1 / tie 0.5 / loss 0`
MBPP	sandboxed execution against unit tests	`pass@1 over k=5`

GRADER VALIDATION GATE

Hand-check 50 graded items per grader and require ≥95% agreement with your own judgment before trusting any downstream number. Bad graders have killed more routing projects than bad classifiers.

$$\hat{Q}(\tau) = (1/n)\ \Sigma_i\ [\ s_{L,i} \cdot 1\{G_i > \tau\}\ +\ s_{S,i} \cdot 1\{G_i \le \tau\}\ ]$$

Cost metric

```
c_L(q) = ( p_in · T_in(q) + p_out · T_out(q) ) / 1e6
```

Report **two** cost models — this is an honesty requirement, not a nicety.

- **M1 — pure API spend**, `c_S = 0`. The headline savings number, and the one the field quotes.
- **M2 — fully loaded**: `c_S = (GPU $/hr) / (queries/hr)`, plus router inference, plus the wasted `S` prefix on escalated queries in cascade variants. **Compute the crossover explicitly** — below some throughput, "free" local inference costs more than Haiku at $1/MTok. That crossover is a real finding.

CURRENT ANTHROPIC LIST PRICES · USD PER 1M TOKENS · AUG 2026

MODEL	INPUT	OUTPUT	ROLE
Claude Opus 5	$5	$25	upper-bound `L`
Claude Sonnet 4.6 / 4.5	$3	$15	`recommended L` matches your assumed pricing
Claude Sonnet 5	$2	$10	cheaper `L` alternative
Claude Haiku 4.5	$1	$5	middle tier in a 3-tier cascade

TWO CORRECTIONS TO THE SPEC

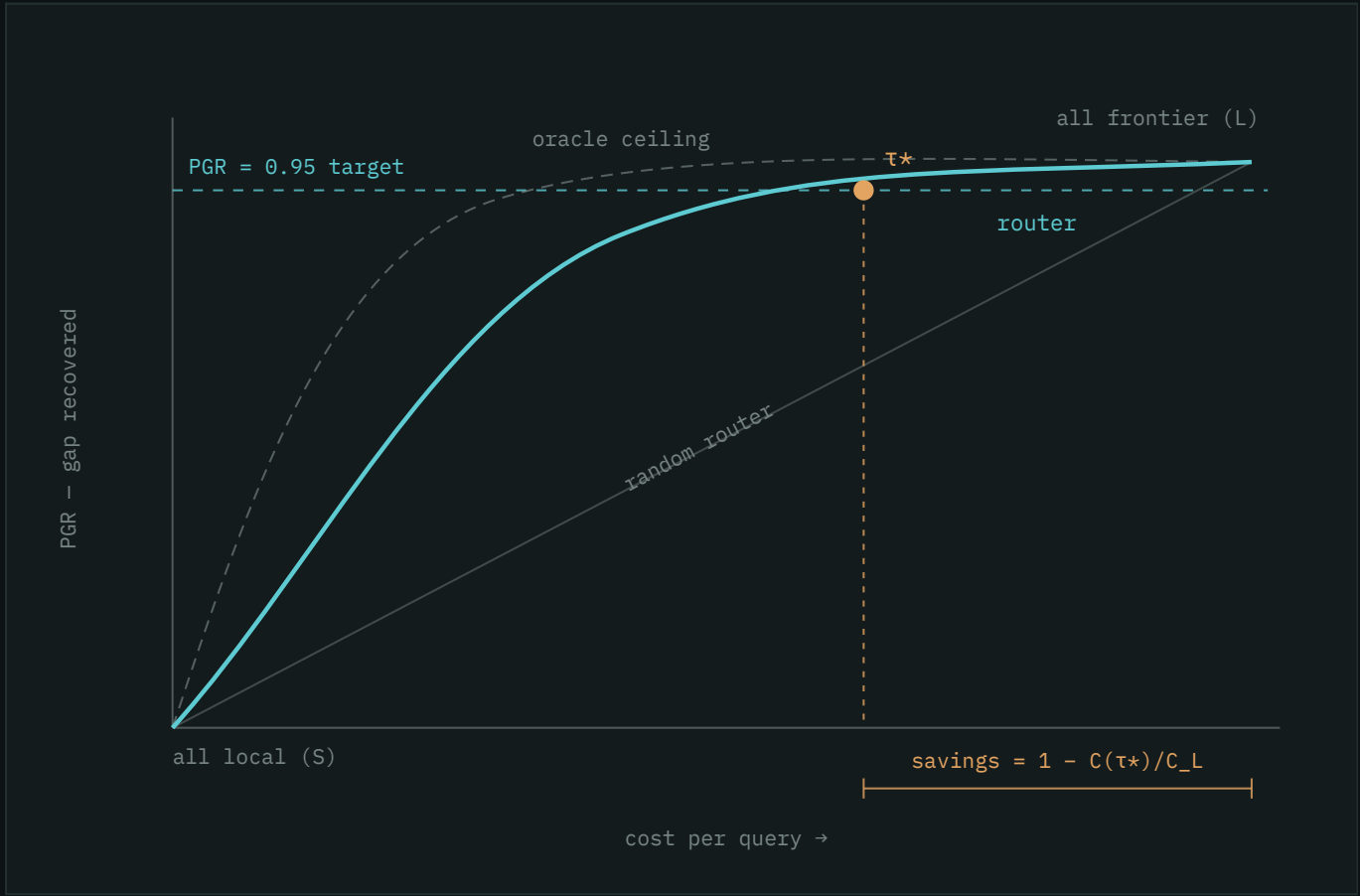
Claude 3.5 Sonnet is retired. The `$3 / $15` you assumed is the current **Sonnet 4.6 / 4.5** rate — use Sonnet 4.6 as `L` and your cost model is unchanged.

Baseline fairness. The Batch API is 50% off and prompt-cache reads are 0.1× input price. A router that beats a *naive* all-frontier baseline 3× may only beat a batch-optimized baseline 1.5×. Report savings against both.

Headline metrics

Adopt the RouteLLM metric set so the results are directly comparable to published work. `α` is the fraction of queries sent to `L`.

```
PGR(τ)    = ( Q(τ) − Q_S ) / ( Q_L − Q_S )      performance gap recovered
CPT(95%) = min{ α : PGR ≥ 0.95 }               ← exactly your project target
APGR      = ∫₀¹ PGR(α) dα                      single scalar, router vs router
Savings   = 1 − C(τ*) / C_L     at PGR = 0.95
```



Every Pareto plot must carry both reference curves. The random-router diagonal is the floor — a router below it is worse than a coin flip. The oracle curve (routing on the true `Δ`, not `Δ̂`) is the ceiling, and the router-to-oracle gap tells you whether to invest in better features or accept irreducible label noise. A router curve shown alone is uninterpretable.

Computing τ* — the math

Work on the calibration split `D_cal`, `i = 1..n`.

STEP 1 · EXACT SWEEP, O(N LOG N), NO SEARCH

$\hat{Q}(\tau)$ and $\hat{C}(\tau)$ are step functions that change only at the n observed values of G. Sort by G descending, compute prefix sums of s_L and suffix sums of s_S (same for costs), and read off $(\hat{Q}, \hat{C})$ at every breakpoint exactly. There are at most n candidate thresholds — enumerate all of them rather than searching.

STEP 2 · MONOTONIZE

Because $\Delta\hat{}$ is imperfect, $\hat{Q}(\tau)$ can be locally non-monotone. Take the **upper-left concave envelope** of the $(\hat{C}, \hat{Q})$ point cloud — a Graham-scan style upper hull. Points strictly inside the hull are dominated and never optimal. Equivalently, isotonic-regress $\hat{Q}(\cdot)$. The envelope *is* the Pareto frontier you plot.

STEP 3 · NAIVE SELECTION

$$\tau_{naive} = \max\{ \tau : \hat{Q}(\tau) \geq Q_{target} \}, \quad Q_{target} = Q_S + 0.95 \cdot (Q_L - Q_S)$$

Both $\hat{Q}$ and $\hat{C}$ are non-increasing in τ, so the cheapest feasible policy is the *largest* feasible threshold.

STEP 4 · CONSERVATIVE SELECTION THIS IS WHAT MAKES THE RESULT REAL

$\hat{Q}(\tau_{naive})$ is optimistically biased because τ was selected to maximize it. Replace the point estimate with a one-sided lower confidence bound:

$$\tau^* = \max\{ \tau : LCB_{1-\delta}(Q(\tau)) \geq Q_{target} \}, \quad \delta = 0.05$$

LCB METHOD	FORM	TIGHTNESS
Hoeffding	$\hat{Q} - \sqrt{ \ln(1/\delta) / 2n }$	Distribution-free but loose. At $n=1500, \delta=.05$ the slack is ≈ **0.032** — 3.2 quality points, often larger than the entire $Q_L - Q_S$ headroom.
Empirical Bernstein / Wilson	variance-adaptive	Typically 2–3× tighter when scores sit near 0 or 1 — which MMLU and GSM8K do.
Paired bootstrap recommended	5th pct of B=2000 resamples	s_L and s_S are measured on the *same* queries, so bootstrap the per-query **difference** $d_i(\tau) = s_{r_\tau(q_i)},i - s_{L,i}$. Paired variance is far smaller — commonly 3–5× tighter than Hoeffding.

Multiplicity correction. τ is selected over m breakpoints simultaneously, so a per-τ bound doesn't control actual risk. Either use the union bound (δ/m, simple and conservative), or — better — note that the family is *monotone* in τ and apply **fixed sequential testing** (Learn-then-Test / RCPS): test thresholds from most conservative to most aggressive and stop at the first rejection. This controls family-wise error at δ with no Bonferroni penalty. It is the correct tool and a strong differentiator for the writeup.

> SIZING IMPLICATION
>
> Bound width is set by `n_cal`. Budget **1,000–2,000 calibration queries per dataset** you intend to quote a guarantee on. With 300, do not claim a guarantee — report the point estimate with a CI and say so.

STEP 5 · REPORT ON TEST, ONCE

Apply the frozen `τ*` to the untouched test split. Report `Q_test`, `PGR_test`, `C_test`, the savings ratio, and the realized frontier-call rate `α_test`, each with a paired-bootstrap CI. Expect `PGR_test < PGR_cal` — the size of that drop is itself a result worth reporting.

Plot spec

- **Panel A** — cost ($/1k queries, log x) vs PGR. Router v1, router v2, random, oracle. Dashed line at PGR 0.95, marker at `τ*`, savings multiple annotated at the intersection.
- **Panel B** — `α` (% calls to frontier) vs PGR. This is the CPT plot.
- **Panel C** — small multiples, one per dataset, same axes.
- **Panel D** — distribution of `Δ̂` split by whether `S` actually succeeded. The visual sanity check that the classifier separates at all.
- Shaded paired-bootstrap 95% bands on every curve.

04 Phased implementation roadmap

File structure

```
cost-aware-router/
├── configs/
│   ├── models.yaml            # model ids, price table, decode params
│   ├── datasets.yaml          # dataset ids, sample counts, split seeds
│   └── experiment.yaml        # feature config, τ targets, δ, bootstrap B
├── router/
│   ├── schema.py              # Query / Response / Score dataclasses + parquet IO
│   ├── data.py                # download + normalize D1..D4 → corpus.parquet
│   ├── dedup.py               # MinHash LSH + embedding dedup → group_id
│   ├── splits.py              # grouped 60/20/20, leave-one-subject-out, nested CV
│   ├── generate.py            # run S and L over corpus (resumable, hash-cached)
│   ├── graders.py             # exact_match, mcq_logprob, llm_judge, exec_test
│   ├── score.py               # apply graders → scores.parquet
│   ├── features.py            # F0 heuristics, F1 tfidf, F2 embeddings, F4 logprobs
│   ├── models.py              # LogReg / TwoHeadMLP / cross-encoder
│   ├── calibrate.py           # isotonic per head + reliability diagrams
│   ├── costs.py               # token counting × price table → $ per query
│   ├── threshold.py           # exact sweep, concave envelope, LCB, LTT selection
│   ├── eval.py                # Q, C, PGR, CPT, APGR, paired bootstrap CIs
│   └── serve.py               # FastAPI /route          (optional, Phase 8)
├── scripts/
│   ├── 00_build_corpus.py
│   ├── 01_acquire_routerbench.py    # was 01_generate_responses.py (Rev A)
│   ├── 02_score_responses.py
│   ├── 03_train_router.py
│   ├── 04_select_tau.py
│   └── 05_plot_pareto.py
├── plots/plot_pareto.py
├── tests/                     # graders, cost math, τ solver, envelope, split integrity
├── artifacts/                 # *.parquet, checkpoints, manifest.json  (gitignored)
├── report/
└── DESIGN.md
```

DATA CONTRACTS BETWEEN STAGES

Each stage reads one parquet and writes one. No hidden state; every stage independently re-runnable.

```
corpus.parquet      : qid, source, group_id, split, prompt, reference, meta
responses.parquet   : qid, model, sample_idx, text, n_in, n_out, logprobs, latency_ms
scores.parquet      : qid, model, score, grader, grader_version
features.parquet    : qid, feature_set, vector
predictions.parquet : qid, s_hat_S, s_hat_L, delta_hat, c_hat_L
```

Execution order & phase gates

PHASE 0 0.5 d	**Scaffolding & cost model.** `costs.py`, `schema.py`, unit tests. Price table read from `models.yaml`. Dry-run spend estimator.	GATE You can print projected `$` for the full generation run *before spending anything*.
PHASE 1 1 d	**Corpus.** `data.py`, `dedup.py`, `splits.py`. Pilot corpus of 2,000 queries — 800 GSM8K, 800 MMLU, 400 Arena.	GATE Dedup report emitted; assert zero `group_id` appears in two splits.
PHASE 2 1–2 d	**Response generation — the expensive, irreversible step.** Content-hash cache, resumable. k=5 from `S`, k=3 from `L`. Use the **Batch API for `L` at 50% off** — this is precisely the offline use case it exists for.	GATE `responses.parquet` complete plus `manifest.json` with model versions, seeds, and actual spend.
PHASE 3 0.5 d	**Scoring.** `graders.py`, `score.py`. Hand-validate 50 items per grader.	KILL-CHECK Compute `Q_S` and `Q_L` per dataset. If `Q_L − Q_S < 0.05`, that dataset has **no routing headroom** — drop it or pick a smaller `S`. Catch this now, not in Phase 6.
PHASE 4 1 d	**Baselines & features.** `features.py`; F0 → F1 → F2 in order. Grouped CV, AUC and Spearman ρ per tier.	GATE F2 beats F0 by more than seed noise. If not, the signal isn't in the query text — revisit the model pair or add F4.
PHASE 5 1 d	**Router training & calibration.** `models.py` two-head training over 5 seeds, then `calibrate.py`.	GATE Reliability diagrams and 15-bin ECE before/after isotonic; validation Spearman reported.
PHASE 6 1 d	**Threshold selection.** `threshold.py`, `eval.py`. Exact sweep, concave envelope, paired bootstrap, Learn-then-Test.	GATE `CPT(95%)` and savings-at-PGR-0.95 on the **untouched** test split, with CIs.
PHASE 7 1 d	**Plots & report.** `plot_pareto.py`, the ablation table, and an error analysis of the 20 costliest routing mistakes.	GATE Pareto figure carrying both random and oracle reference curves. Report drafted.
~~PHASE 8~~ ~~optional~~	**Serving.** ~~FastAPI `/route`, shadow-mode logger, latency measurement.~~ **Dropped in Rev A** — thesis target, no serving requirement.	GATE n/a — phase removed.

> SCALE-UP DECISION POINT
>
> Run Phases 1–7 end to end on the 2,000-query pilot *first*. Only once the pipeline produces a valid Pareto curve should you expand to the full ~10k corpus. The pilot costs a few dollars; a broken full run costs a weekend.

The ablation table to fill in

This is the core result of the report. Everything above exists to populate these six rows.

CONFIG	AUC	P(Δ̂,Δ)	CPT(95%)	SAVINGS @ PGR .95	APGR
Random router	0.50	0.00	0.95	~5%	0.50
F0 heuristics + LogReg	·	·	·	·	·
F1 TF-IDF + LogReg	·	·	·	·	·
F2 embeddings + MLP	·	·	·	·	·
F2 + F0 `headline`	·	·	·	·	·
+ cost-normalized rule (v2)	·	·	·	·	·
+ F4 logprob cascade	·	·	·	·	·
Oracle router	1.00	1.00	·	·	1.00

05 Risks & expectation-setting

RISK	MITIGATION
No headroom. If `S` and `L` score similarly, no router helps. A *strong* small model is bad news on easy benchmarks — Phi-4-mini reportedly scores ~88.6% on GSM8K, leaving ~7 points against a frontier model.	Phase 3 kill-check. MMLU (~67% vs ~88%) and open-ended Arena traffic have far more room.
Target ambiguity — absolute-95% and PGR-95% differ enormously.	Fixed in §0. Commit before generating anything.
Overfit τ.	Three-way split, paired-bootstrap LCB, Learn-then-Test (§3).
Unfair baseline. Batch API and prompt caching already cut frontier spend 50–90%.	Report savings against both an unoptimized and a batch-optimized frontier baseline.
Distribution shift. Benchmarks are not production traffic.	Arena is the realism proxy. Never average it with GSM8K/MMLU.
Judge self-preference when the judge shares a family with `L`.	Cross-family judge, position swap, report agreement rate (§2).
Local serving economics. At low throughput, "free" local inference costs more than Haiku 4.5.	Compute the crossover explicitly under cost model M2.

What "2–5×" realistically means

RouteLLM, routing between GPT-4-Turbo and Mixtral-8x7B, reported at 95% performance: **~85%** cost reduction on MT-Bench, **~45%** on MMLU, **~35%** on GSM8K.

> **Calibrate expectations accordingly:** 2–5× savings is realistic on open-ended, Arena-like traffic. *On GSM8K alone, expect closer to 1.5×.* Put the headline number on a blended or Arena-weighted traffic mix, and state the mixture weights explicitly rather than quietly picking the flattering one.

Stretch: the three-tier cascade

`local → Haiku 4.5 ($1/$5) → Sonnet 4.6 ($3/$15)`. Given Haiku's price point, the two-tier local-vs-Sonnet framing may well be dominated by a three-tier one. The two-head architecture extends to `k` heads with no redesign — add `ŝ_Haiku` and solve the two-threshold version of the same constrained problem. Good future work, or a genuine result if time allows.

06 Delivery — merging the two roadmaps, and splitting four ways

Verdict on `implementation_plan_2.md`: **right shape, wrong statistics.** Roughly 60% of it survives intact — keep its scaffolding, its stack and its scope discipline; replace its labelling, splitting and threshold logic. It is a good plan for building *a* router and a weak plan for defending *a result*.

Keep — including two things it gets right that this document did not

FROM PLAN 2	WHY IT STAYS
"Get the data without generating it" (its Phase 2)	The same conclusion as Rev A, reached independently. Strong instinct — this is the decision the whole project hinges on.
Dual-stream structural + semantic features	Exactly the F0 + F2 headline config in §1.3. Same answer, arrived at from a different direction.
XGBoost baseline `adopt`	Not in this document, and it should have been. Gradient boosting on fused structural + embedding features is a strong, cheap tabular baseline — it belongs in the ablation ladder between F1 and F2, and it will occasionally beat the MLP on this data size.
Concrete stack, `requirements.txt`, flat `src/`	Lower ceremony than §4.1. A four-person team can start on Monday with it. Keep the shape; keep this document's module boundaries inside it.
FastAPI demo (its Phase 6) `reinstate`	Rev A dropped serving because the target is a thesis. With four people it is affordable again, and a live routing demo with a running cost counter is worth a great deal at a defence. **Phase 8 comes back.**

Fix — eight defects, in descending order of how much they would cost you

#	DEFECT IN PLAN 2	FIX
1	**The label collapses.** `Y = 1 if small ≥ 0.90 × large`. On GSM8K and MMLU per-item scores are 0 or 1, so the rule reduces to *"small correct OR large wrong"* — the 0.90 does nothing, and the ratio is undefined when the large model scores 0. Worse, binarising at training time discards the magnitude that the threshold sweep needs.	Regress the gap `Δ = s_L − s_S` with two heads (§2.1). Never binarise.
2	**τ is swept on the test set.** Its Phase 5.2 sweeps thresholds over "the Test Set" — the same data the savings are reported on. The headline number is guaranteed optimistic, and an examiner who knows this field will find it.	Three-way split; τ selected on `cal` only, test opened once (§2.5).
3	**No dedup, no grouped split.** A random 80/20 over Arena-style data leaks near-duplicate prompts across the boundary.	MinHash + embedding dedup, split by connected component (§2.5).
4	**The Pareto plot has no reference curves.** Without the random and oracle brackets you cannot tell whether the router beat a coin flip — which is the only question the figure exists to answer.	§3.3 — random floor and oracle ceiling on every plot.

#	DEFECT IN PLAN 2	FIX
5	**No headroom kill-check.** If `Q_L − Q_S` is small, nothing works — and you find out in its Phase 5, after all the work is done.	§4 Phase 3 gate, run the moment scores exist.
6	**No cost model.** "Calculate Total Cost" with no token accounting and no price table is where quiet arithmetic errors live.	`costs.py` first, with tests (§4 Phase 0).
7	**Ollama + Llama 3 8B on an M2 Air** (~13 tok/s, dated model), and `openai` as the large arm with no budget.	Appendices A and B.
8	**The notebook is the home of the headline result.** Notebooks are for exploration; an untested notebook cell should not be what your thesis number depends on.	`threshold.py` as a tested module; the notebook calls it.

Net: plan 2's Phases 1–4 survive with fixes 1, 3 and 6 applied. Its Phase 5 is replaced wholesale by §3. Its Phase 6 returns as Phase 8.

Splitting four ways

THE PRINCIPLE

Split by artifact, not by phase. Phases are sequential — four people assigned to sequential phases means three people idle. The seams are the parquet contracts, and §4.1 already defines them. Each person owns a module boundary and the file format that crosses it.

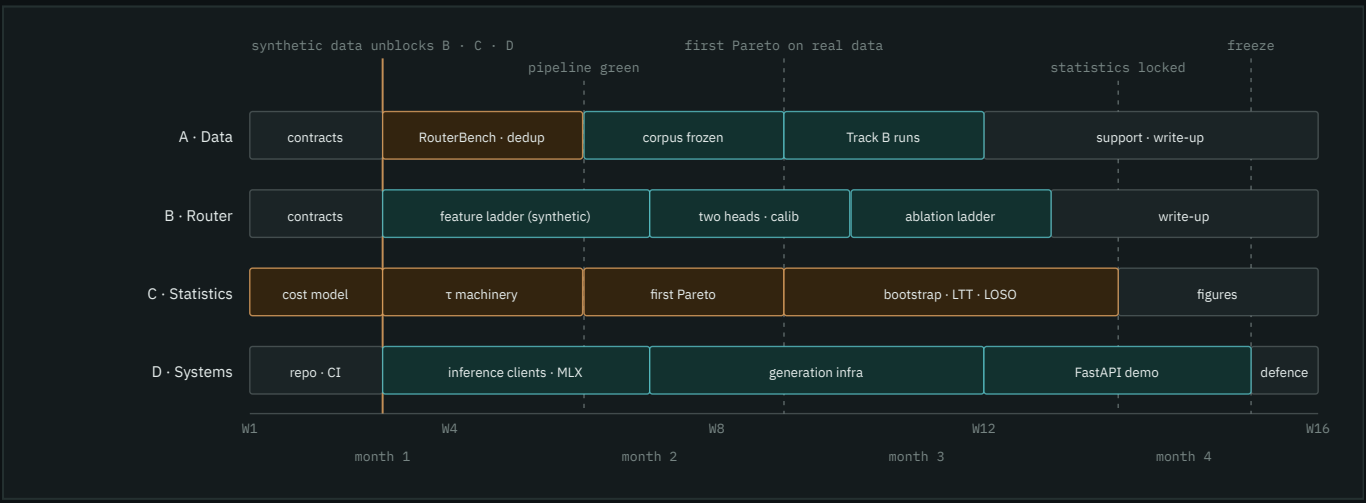
THE SCHEDULING TRICK THAT MAKES IT ACTUALLY WORK

In week 2, generate a **synthetic `scores.parquet`** with a known planted signal — a difficulty variable that drives both `s_S` and `s_L` with a controllable gap. From week 3 everyone downstream develops and tests against it, before any real data exists. The evaluation module can be finished and unit-tested *entirely* on synthetic data, where you know the right answer. This defuses the classic four-person failure mode: the statistics person starting last and the whole project jamming in the final fortnight.

ROLE	OWNS THESE MODULES	OWNS THIS CONTRACT	THESIS CHAPTER
A · Data & Corpus `most careful person`	`data.py` · `dedup.py` · `splits.py` · `01_acquire_routerbench.py`	`corpus.parquet` `scores.parquet`	Datasets, deduplication, and split methodology
B · Features & Router	`features.py` · `models.py` · `calibrate.py` · `graders.py`	`features.parquet` `predictions.parquet`	Feature engineering and router architecture
C · Evaluation & Statistics `strongest stats person`	`costs.py` · `threshold.py` · `eval.py` · `plot_pareto.py`	the Pareto figure the ablation table	Cost modelling, threshold selection, and distribution-free guarantees

ROLE	OWNS THESE MODULES	OWNS THIS CONTRACT	THESIS CHAPTER
D · Systems & Demo	`generate.py` · `serve.py` · CI · `manifest.json`	the inference layer reproducibility	Local inference, serving architecture, and the live prototype

C is the role to staff first. Everything in §3 — the exact sweep, the concave envelope, the paired bootstrap, Learn-then-Test — *is* the project's contribution. A bug in the τ solver silently invalidates every number in the paper, and nobody downstream will catch it. Give it to whoever is strongest at statistics and most willing to write tests. A is the second-most consequential: dedup and split bugs are invisible and fatal.



Amber marks the critical path. The corpus lane and the statistics lane are what everything else waits on; the orange line at week 2 is the synthetic-data handoff that lets three lanes run in parallel for the next six weeks without any real data existing. If a week slips, it must not slip in an amber block.

Gates — what "done" means at each marker

WEEK	GATE	EVIDENCE
2	**Contracts frozen**	All five parquet schemas agreed and in `schema.py` ; `costs.py` passing tests; synthetic `scores.parquet` with a planted signal committed. Nobody is blocked after this point.
5	**Pipeline green on synthetic**	End-to-end run on fake data produces a Pareto curve. It will be nonsense — but the plumbing, the τ solver and the plot code are all proven before real data arrives.
8	**First Pareto on real data**	RouterBench pair chosen by headroom; `Q_L − Q_S` reported per dataset; the §4 Phase 3 kill-check passed. **If this slips, cut scope, not rigour.**
13	**Statistics locked**	`CPT(95%)` and savings on the untouched test split, with paired-bootstrap CIs and the Learn-then-Test bound. No number changes after this week.
15	**Freeze**	Demo running, figures final, draft complete. Week 16 is buffer and defence rehearsal — not work.

Four rules that keep a four-person ML project from failing

1. **Rotate an integrator weekly.** One named person merges to `main` and runs the full pipeline end to end every Friday. The role rotates so all four understand the whole system by the defence.

2. **Artifacts are not git.** Parquets and checkpoints are gitignored — put them in a shared Drive folder or a private Hugging Face dataset repo, named by content hash. Never emailed, never renamed by hand.
3. **The frozen-corpus rule is a team rule.** Nobody regenerates responses without bumping the manifest version. One silent regeneration mid-project and every comparison made before it becomes meaningless.
4. **Every owner writes their own tests** — and C's matter most. A wrong τ solver produces plausible, publishable, entirely false numbers, and no downstream person will notice.

A Choosing the local model S

Two constraints decide this, and neither is the one people reach for first.

> CORRECTION TO THE REV A FIGURE
>
> The "25–40 tok/s" quoted in Rev A is an M3/M4-class number. **The M2 base chip has ~100 GB/s of memory bandwidth**, and decode speed on a bandwidth-bound machine is roughly `bandwidth ÷ model bytes`. A 9B at 4-bit therefore runs nearer **~13 tok/s**, not 30 — and the fanless Air throttles 15–25% under sustained multi-hour load. Every wall-clock estimate below is rebuilt on the corrected number.

Constraint 1 — bandwidth, not memory, is the ceiling

16 GB is plenty to *load* a 9B model. It just runs slowly. The binding resource for a project that needs thousands of completions is tokens per second, and on an M2 that is set by bandwidth divided by how many bytes the weights occupy. Halving the model roughly doubles the throughput.

Constraint 2 — you want `S` weak, not strong

This is the counterintuitive one. The router only has something to learn when `Q_L − Q_S` is large (§5, risk 1). A small model that is *already good* at your benchmark destroys the experiment. So the pressure from both constraints points the same direction: **down the size ladder, not up.**

MODEL	4-BIT SIZE	EST. TOK/S	TRACK B GSM8K	LICENSE	WHY THIS ONE
Qwen3.5-4B `primary`	~2.5 GB	35–40	~9 h	Apache 2.0	Modern, permissive, and fast enough for one overnight run with memory left over for the bge encoder and the dedup pass. The default pick.
Llama 3.2 3B	~2.0 GB	45–50	~7 h	Llama CL	**Maximum headroom, fastest turnaround.** Weakest of the set (~63% MMLU, ~78% GSM8K), so `Q_L − Q_S` is largest and the routing signal is cleanest. Best-documented tooling of any small model.
Phi-4-mini 3.8B	~2.3 GB	35–40	~9 h	MIT	**The contrast case.** ~88.6% on GSM8K nearly closes the math gap on its own. Run it *deliberately* to demonstrate risk 1 empirically rather than only asserting it. Cleanest license of the five.
Gemma 3 4B	~2.8 GB	30–35	~10 h	Gemma	**Robustness column.** Different pretraining lineage, 128K context, vision-capable — checks that your findings are not an artifact of the Qwen family.

MODEL	4-BIT SIZE	EST. TOK/S	TRACK B GSM8K	LICENSE	WHY THIS ONE
Qwen3.5-9B	~5.5 GB	~13	~24 h	Apache 2.0	**Upper bound — only with a cut corpus.** Loads comfortably in 16 GB, but the wall-clock kills a full Track B. Use it on a 400-item subset to show the headroom trend continues.

> TURN RISK 1 INTO A RESULT — THE HEADROOM SWEEP
>
> Run rows 2, 1 and 3 (Llama 3.2 3B → Qwen3.5-4B → Phi-4-mini) against the same frontier arm and plot `CPT(95%)` against the measured `Q_L − Q_S`. Three overnight runs, no extra spend, and it converts the project's biggest risk into a quantified finding: *how much headroom does a router need before it pays for itself?* That is a better thesis chapter than any single-pair result.

Two levers that beat any model choice

- **Batch your generation.** On a bandwidth-bound machine the weights are read once per batch, so MLX batched decoding at batch 8 gives roughly 3–5× effective throughput. Do this before optimizing anything else — it is worth more than dropping a model size.
- **Or don't run it locally at all.** Serve the same open weights on a free inference tier (Appendix B, row 3) and a 24-hour laptop run becomes ~3 hours. Since `c_S = 0` is a modeling choice in Rev A, not a measurement, nothing in the paper changes.

MEASURE BEFORE YOU PLAN AROUND THESE NUMBERS

```
mlx_lm.generate --model mlx-community/Qwen3.5-4B-Instruct-4bit \
  --prompt "Solve: 17*23. Think step by step." --max-tokens 256
```

Record the reported tok/s in `manifest.json`. Every wall-clock figure in the table is derived from memory bandwidth, not measured on your machine — one command replaces all of them with facts.

B Choosing the API arm

THE MOVE THAT MAKES A ZERO-BUDGET THESIS RIGOROUS

The model you *run* and the model you *price* do not have to be the same model. Rev A already re-prices logged RouterBench responses at 2026 rates, so the cost axis is a modeling layer, not a receipt. Run the experiment on free tiers; price `c_L` at frontier list rates; state the separation plainly in Methods. That is a defensible design, not a compromise — and it is the only way this project works at $0.

PROVIDER / MODEL	FREE TIER	LIST PRICE /1M	ROLE HERE
Google Gemini 3 Flash · 3.5 Flash `primary free arm`	Yes — AI Studio, ~15 RPM / ~1,500 RPD, no card	$0.50 / $3.00 $1.50 / $9.00	**The frontier arm for Track B.** 1,500 requests/day covers a full 1,500-item GSM8K pass in one day, free. Caveats to disclose: free-tier prompts may be used to improve Google's products, and Pro models are paid-only.
GitHub Models GPT-5 / GPT-4o class	Yes — GitHub account, ~10–15 RPM / 50–150 RPD	—	**Quality anchor.** The daily cap is low, but these are genuinely frontier-class models at $0. Use a 200-item subset to verify that Flash-derived labels track a true frontier model. Your Student Developer Pack raises the limits.
Groq · Cerebras llama-3.1-8b · qwen3-32b · gpt-oss	Yes — 14,400 RPD, 6,000 TPM (Cerebras ~30–60k TPM)	—	**Serve the small arm, not the frontier one.** These host open weights, so they are not a frontier model — but 6,000 TPM turns a 24-hour laptop run into ~3 hours, and they are the natural middle tier for the three-tier cascade in §5.
Anthropic Sonnet 4.6 · Haiku 4.5	No	$3 / $15 $1 / $5	**The pricing reference.** Keep Sonnet 4.6 as the `c_L` constant — it matches your original spec and is a credible frontier price. If budget appears, ~$2 buys 1,000 GSM8K items on the Batch API at k=1.
OpenAI GPT-5.6 Terra · GPT-5 nano	No	$2.50 / $15 $0.05 / $0.40	**Cheapest paid escape hatch.** 1,000 GSM8K items ≈ $2 on Terra via Batch, or ~$0.11 on nano. Nano is far too weak to serve as `L`, but it is an excellent extra *middle* tier for the cascade.

The recommended stack

Track A — primary, $0	RouterBench. No API call at any point.
Track B — small arm	Qwen3.5-4B at 4-bit via MLX, batched. Fall back to Groq free if wall-clock bites.
Track B — frontier arm	Gemini 3 Flash free tier for volume; GitHub Models on 200 items as the quality anchor.
Cost model	`c_L` priced at Sonnet 4.6 ($3 / $15), `c_S = 0`. Report the run-vs-price separation explicitly.

READ THE TERMS BEFORE YOU BUILD ON A FREE TIER

Free tiers differ in ways that matter for a submitted thesis. Cohere's trial keys are **non-commercial only** (fine for academic work — but say so). Google's free tier permits training on your prompts. Cerebras' free catalogue has churned before. None of these is disqualifying; all of them belong in a footnote, and none should be discovered the week before submission.

C Answered — and what each answer changed

Local compute → Apple Silicon, 16 GB+

`S` = Qwen3-8B at 4-bit via MLX. MMLU scores by logprob in minutes; GSM8K generation becomes an overnight job. F3 fine-tuning is feasible on MPS now that RouterBench removes the data limit.

API budget → none

The single biggest change. Phase 2 becomes acquisition rather than generation: RouterBench supplies the frontier arm for $0, and re-pricing it with `c_S = 0` turns the constraint into the thesis's contribution.

Goal → academic report / thesis

Phase 8 dropped. Benchmark-first, Arena demoted to optional, and the reporting weight moves onto the τ statistics, the LOSO split, and the ablation ladder.

Still open · judge model

Moot for Track A — MT-Bench in RouterBench is pre-judged, so there is no judge to bias. It returns only if you add fresh open-ended evaluation later.

References — Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data* (2024): PGR / CPT / APGR metrics · Angelopoulos et al., *Learn then Test* & *Risk-Controlling Prediction Sets*: distribution-free threshold selection · Chen et al., *FrugalGPT*: cascade framing · Ding et al., *Hybrid LLM*: quality-gap regression routing · *RouterBench*: reporting conventions.

Prices verified against **platform.claude.com/docs**, August 2026. Source of truth for this document is `DESIGN.md` in the project repo.